Generating the Deliverable Set from a Resolved Deployment

Project Task 3: Code Generation
Project Type: Code Generation
October 30, 2026

Joris Jonkers (s1829157)

Model Driven Engineering
University of Twente

Contents

1 Introduction **1**

1.1 The Pipeline . . . 1
1.2 Changes Since Task 2 . . . 2
1.3 Roadmap . . . 2

2 Alternatives and Justification **2**

2.1 Constraints Set by the Course . . . 2
2.2 Overview of the Design Decisions . . . 2
2.3 The Model-to-Text Step . . . 3

3 Solution Design **4**

3.1 Interface: Input, Output and Entry Point . . . 4
3.2 The Main Template and the Path Plan . . . 6
3.3 One Template per Deliverable Kind . . . 6
3.4 Shared Templates and the Whitespace Rules . . . 8
3.5 Writing YAML and JSON . . . 8
3.6 Queries: Formatting and Selection . . . 9
3.7 The Java Class and Failure Handling . . . 9
3.8 Worked Example: One Route from Model to File . . . 10

4 Tests and Evaluation **10**

4.1 Testing Strategy . . . 10
4.2 Example Models . . . 12
4.3 Results . . . 12
4.4 Defects Found . . . 12
4.5 Limitations . . . 13

5 Reflection **13**

5.1 How the Project Changed Over Time . . . 14
5.2 What Went Well . . . 14
5.3 What Could Have Been Better . . . 14
5.4 What Was Learned . . . 15

6 Outlook **15**

A Changes to the Task 1 Metamodels and Task 2 Transformations **16**

A.1 Source Metamodel (Project Intent) . . . 16
A.2 Platform Intent . . . 16
A.3 Pinned Inputs . . . 18
A.4 Target Metamodel (Resolved Deployment) . . . 21
A.5 The Transformations . . . 21

B The Production Implementation **25**

1 Introduction

The project converts short authored documents into files that a cluster's GitOps controller applies. Each Application's repository holds one `Project Intent` document, which states what its `Application`s need: their `Process`es, limits, ports, dependencies and external Routes. One `Platform Intent` document states what is shared across the estate (the Projects and the shared platform). Three read-only inputs are pinned by digest: the node contract, the images lock and a `ClusterState` snapshot. The output is the *Deliverable Set*: Kubernetes objects, custom resources for Flux, Flagger and the Vault Secrets Operator, and Vault configuration documents. Only Flux applies that set.

The authored documents state requirements and facts rather than mechanisms. The derived `Resolved Deployment` records every platform decision. The Deliverable Set serialises those decisions without making further choices (`spec/v1/00-overview.md`). The final transformation only writes the target syntax.

1.1 The Pipeline

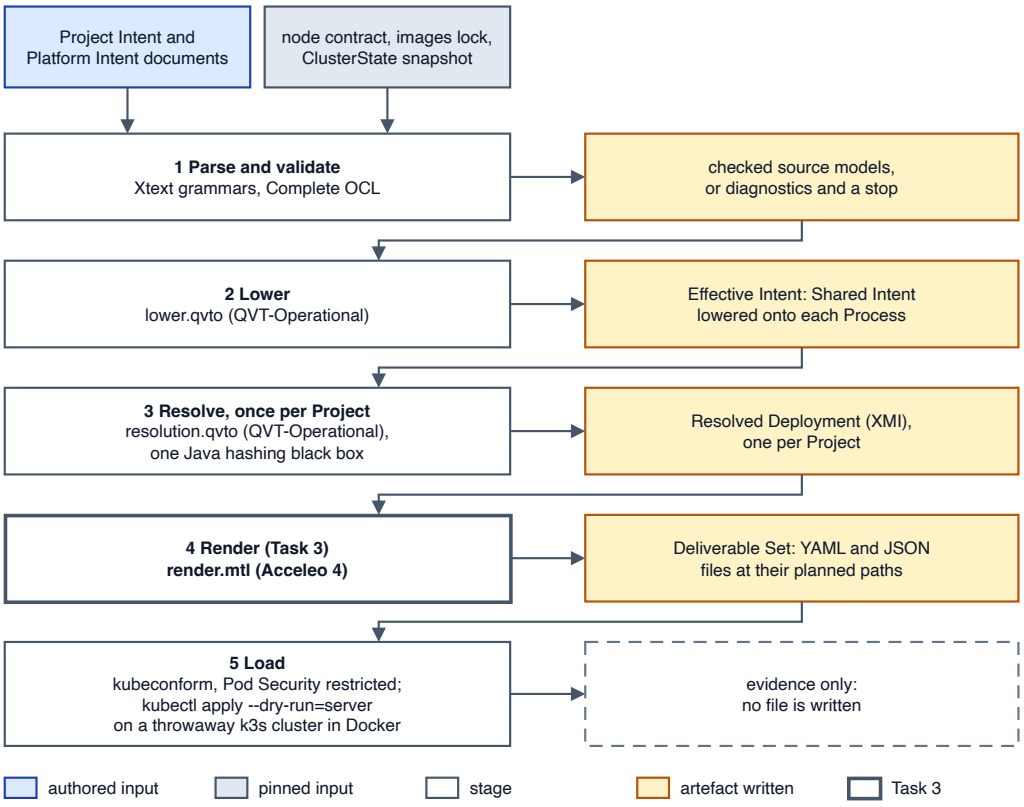


Figure 1: The End-to-End Pipeline

Figure 1 shows five stages; Task 3 covers only rendering. Xtext grammars and Complete OCL constraints parse and validate each document. A QVT-Operational lowering (the flattening in Task 2) copies each Shared Intent declaration onto every `Process` that uses it. A second QVT-Operational transformation resolves each `Project` using the lowered models of every `Project`, the Platform Intent and the pinned inputs. It produces one Resolved Deployment with a path plan that assigns each Deliverable to a file. If resolution cannot derive a value, the run stops. The Acceleo templates then run once over the set of Resolved Deployments. Finally, the load stage checks the written files against pinned schemas, the Pod Security `restricted` profile and a server-side dry run on a throwaway k3s cluster (Section 4).

Only resolution makes decisions; its oracle (the committed expected models) checks them. Rendering is checked by byte comparison with the committed `rendered` trees and by the load stage. A matching render demonstrates correct syntax, but does not establish that the decisions were correct.

1.2 Changes Since Task 2

The model was refined as it was applied to production: deploying the real estate surfaced cases the model had to express, and the changes came from those cases. The estate pass, a third transformation over all Resolved Deployments, was removed. The migration from the previous setup, fleet-infra with its own DSL, was partly manual; what exists only because of it, chiefly moving data between Instances (a Move), stems from the old deployment method rather than the DSL, and is left out of the DSL and of this report. Section A lists each change with its reason.

1.3 Roadmap

Section 2 sets out the choices; Section 3 describes rendering (Figures 2 and 3); Section 4 reports the parity and load evidence (Figure 4); Section 5 reflects on the three tasks (Figure 5); Section 6 looks beyond the course.

2 Alternatives and Justification

Table 1 summarises every decision below.

2.1 Constraints Set by the Course

The course requires Ecore metamodels, QVT-Operational or ATL for model-to-model transformation, Acceleo for model-to-text generation, Eclipse/EMF projects in the archive, and a generated artefact loaded in a tool (here, the Kubernetes API server). The project defines its own domain, target and source syntax. OCL cannot hash canonical bytes, so one Java library computes only that digest. Maven with Tycho builds the projects without an IDE; an examiner can also run the generator from Eclipse.

2.2 Overview of the Design Decisions

Each decision is presented with its strongest alternative and the criterion used to reject it.

2.2.1 Source Model: One Intent per Project, Plus a Platform Intent

Intent is authored in one file per Project, plus one Platform Intent for facts that must be unique across the estate (`docs/adr/model/0009-intent-is-authored-one-file-per-project.md`, `docs/adr/model/0045-platform-intent-is-the-second-authored-document.md`). One file per Application was rejected because each fact needs a single source: the same OpenTelemetry variables would otherwise be repeated for three Processes. Lowering merges shared declarations onto each Process, so templates need not combine information from different levels.

2.2.2 Transformation Chain: Lower, Resolve, Render

Three typed steps replace a single transformation from source to Resolved Deployment. A single transformation was rejected because it obscures the source of faults: each intermediate model has an oracle that identifies which step produced a fault. Rendering reads only the Resolved Deployment and therefore only writes text.

2.2.3 Removing the Estate Pass

The Task 2 pipeline had a third transformation over every Resolved Deployment. It was removed because each Project owns and alone writes every object it delivers (decision record 0116 under `docs/adr/model/`). Keeping it would break the delivery guarantee that publishing an unrelated change deploys nothing: one policy Job held every Project's grants, so one change re-ran it for all, and one fault left seven Projects not ready. Every generated file now belongs to one Project's Resolved Deployment, so no template merges output across Projects.

2.2.4 Kubernetes Objects and Custom Resources as the Target

The target consists of Kubernetes API objects and custom resources for Flux, Flagger, the Vault Secrets Operator, Traefik and the Prometheus Operator. Flux is the only applier, and it pulls the render by digest (`docs/adr/model/0050-delivery-is-part-of-the-model.md`). A push path from CI was rejected because it

would require a deployment credential outside the cluster. Helm or Kustomize templating was rejected because no abstract model could express its constraints (Task 1 report). Each generated file represents an object that the API server validates; the load stage checks this validation.

2.2.5 One Rendering Run for All Projects

Each Project is resolved separately using the union of all Projects, and rendering runs once over the resulting set. Resolution from a Project's own documents alone was rejected because it cannot account for cross-Project facts: a provider's address, admission of a consumer by label, or rejection when a provider removes a Surface named by a consumer (`spec/v1/40-composition.md`). The cost is that one Project's change can alter another's output through names. A test checks that unrelated Projects' output remains unchanged.

2.3 The Model-to-Text Step

Rendering is the only step that writes text. Its output must match the committed `rendered/` trees byte for byte, including whitespace, key order, quoting and headers.

2.3.1 The Language: Acceleo 4

The assignment prescribes Acceleo, which also meets three technical criteria. Acceleo 4 has a standalone API, allowing one Java class to run the module without an IDE, as in CI. Its whitespace rules give templates exact control over the output bytes. Queries use Acceleo Query Language (AQL) expressions over the Ecore model; the templates are interpreted rather than compiled to Java.

The Epsilon Generation Language (EGL) [2] also runs standalone over Ecore and would meet the first two criteria; the assignment settles the choice. Xtend is a separate JVM language that compiles to Java, not the template language of Xtext. Acceleo implements the OMG standard MOFM2T [1]; no conformance is claimed beyond the constructs the module uses.

2.3.2 Templates Rather Than a Serialiser

The alternative is a serialiser: a visitor writes typed objects through a YAML library and a JSON encoder. The model-driven implementation uses templates without a subsequent formatting pass.

The criterion is that one reviewed artefact determines the output bytes. A template directly specifies the file layout, so a difference from the oracle points to a visible template line. A serialiser delegates key order and quoting to library defaults; a formatting pass adds a second tool whose version affects the bytes. Comparing parsed documents would not check the headers, ordering or quoting specified by the committed trees.

Two further alternatives were rejected:

- A model-to-model step into a Kubernetes object metamodel adds a third metamodel of 44 object kinds that repeats the Resolved Deployment's shape, and the target syntax still has to be written.
- The Xtext serialiser needs a grammar and a formatter per kind, and gives no control of the header line or key order without a custom formatter.

The cost is manual whitespace control: the template must specify every layout byte (Section 3.4). The template also writes the key order of the Vault JSON documents, which have no header. A string is double-quoted when a YAML implicit resolver would read it as a null, a boolean, a number or a timestamp, or when it ends in a colon or contains "`: `".

2.3.3 Template Structure: One Module with Dispatch by Type

MOFM2T lets a module import others and extend or override their templates. This is useful when a generator varies by platform. Here there is one target and no variant to override; shared templates (label set, peer selector, consumer label) serve nearly every kind. Splitting by kind would require imports throughout; one module retains a single namespace. The cost is a large file navigated by template name: 3772 lines holding 113 templates and 33 queries.

YAML generation calls one overloaded `objects` template, which Acceleo resolves by the Deliverable's runtime type (a polymorphic template in the course's terms). It replaced a chain of 22 type tests with no `else`, where a kind without a branch produced only the header. A kind without a template now causes the run to fail.

2.3.4 Templates Use Only Values Present in the Model

No template derives a value absent from the model. Platform decisions are made in the QVT-Operational transformations and recorded in the Resolved Deployment, so a missing derivation produces a missing or different file. This preserves release identity: a release's revision is computed from the Application's canonical JSON, so a value derived in a template would fall outside that identity.

Templates handle only target-syntax formatting. A duration becomes whole seconds, precedence becomes a Traefik priority of 1000 minus the precedence, and a match becomes `Path` or `PathPrefix`. The templates also apply fixed per-kind rules of the target format: the rolling-update surge, the `Recreate` strategy, the disruption-budget selector and the suspension of migration Jobs. These rules do not vary by model. Queries select and order the objects a template visits without deriving values.

Decision area	Rejected alternative	Chosen approach	Reason
Source	One file per Application	One Intent per Project	One place per fact
Chain	One transformation	Lower, resolve, render	Oracle for each model
estate pass	Keep it	Removed	Unrelated publication deploys nothing
Target	Push path; Helm; Kustomize	Objects pulled by Flux	No external credential
Rendering input	One Project alone	Union, one run	Cross-Project facts
Language	EGL; Xtend	Acceleo 4	Headless API, whitespace
Text output	Serialiser; formatter; parsed comparison	Templates write bytes	One artefact determines bytes
Other options	Model-to-model Kubernetes metamodel; Xtext serialiser	Rejected	Extra metamodel; no byte control
Modules	Imports, overriding	One module	No variant
YAML dispatch	Chain of type tests	Dispatch by type	Missing kind fails
Derivation	Queries in templates	Model holds values	Identity covers values

Table 1: Summary of the Design Decisions and the Rejected Alternatives

Section 4 discusses the remaining limitations and Section 5 the decisions that were reversed.

3 Solution Design

Figure 2 shows the model-to-text step: a main template, one template family per Deliverable kind, the shared templates those families use, and the Java class `Rendering` that loads and runs the module. Section 2.3 justifies the choice of Acceleo 4. The module, `emf/bundles/render/model/render.mtl`, reads the Resolved Deployment produced by the preceding transformations. Section 3.8 traces one Route from model to file (Figure 3).

The design applies four course concepts. Dispatch by kind uses polymorphic templates (Section 3.3). Every run regenerates every file in full, so no protected areas are needed. Each stage is tested on its own against committed files, and the output is held byte for byte to committed oracle files (reference trees that both implementations must match; Section 4.1.1), so no trace blocks map text back to model elements. The JSON templates use text-explicit escape (`['[']`) to write a literal bracket.

3.1 Interface: Input, Output and Entry Point

Input. The step reads one Resolved Deployment for each Project in the run, each containing a `deliverables` list of the files the Project delivers. Lowering and resolution run for each Project in turn; `Rendering` copies the results into one model for a single rendering run. Templates read no files, environment variables or system clock: all other inputs are consumed beforehand and represented as model features. Each Deliverable has a `path` relative to the output root, assigned by the resolution stage. The metamodel has 41 concrete Deliverable kinds (`emf/bundles/metamodel/model/resolved-deployment.ecore`), each with a template family of its own.

Output. The Deliverable Set contains one file per Deliverable at its assigned path, with nothing else under the root. A YAML file begins with one header line and contains one or more objects, each introduced by three

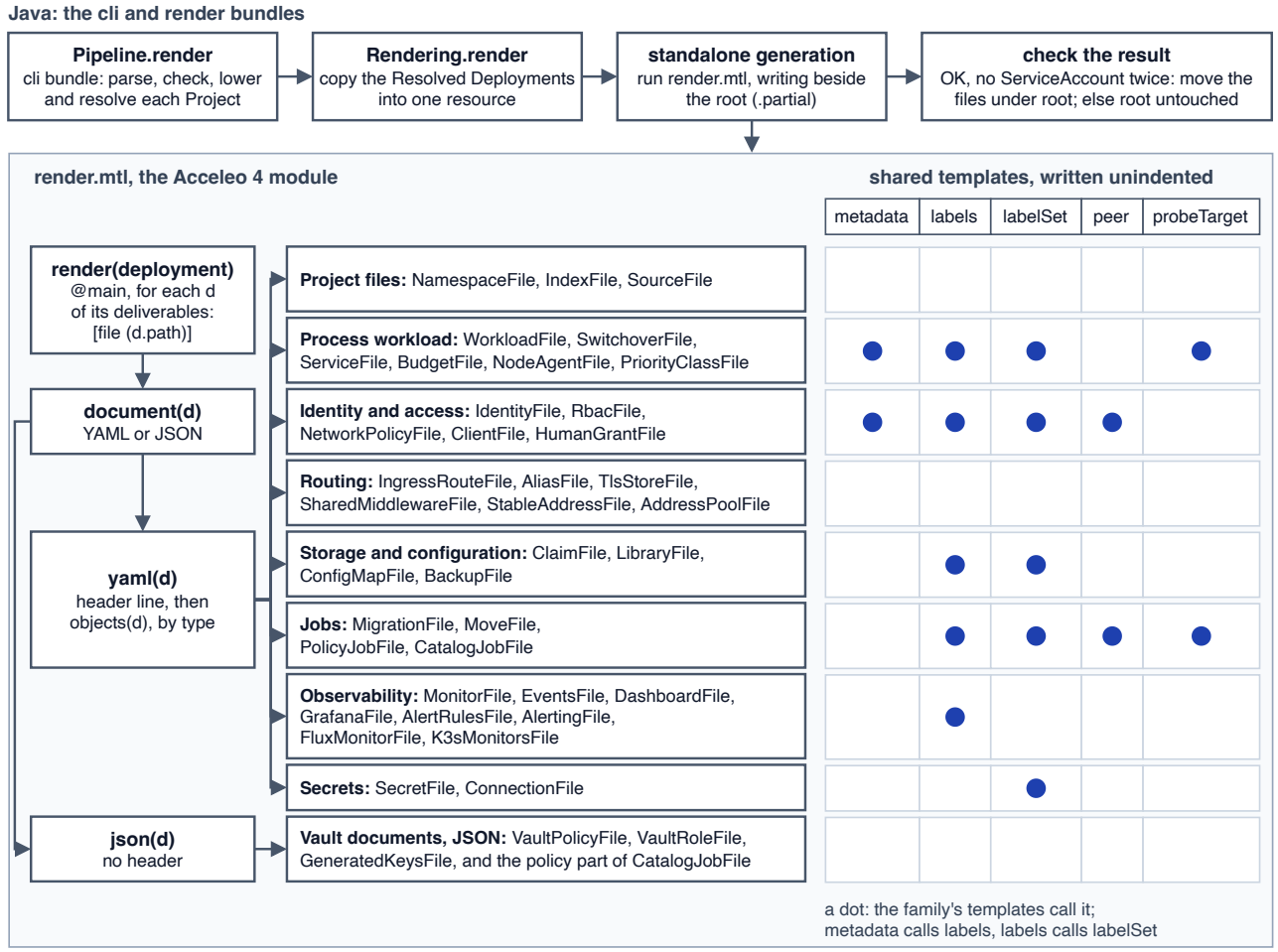


Figure 2: Template Structure

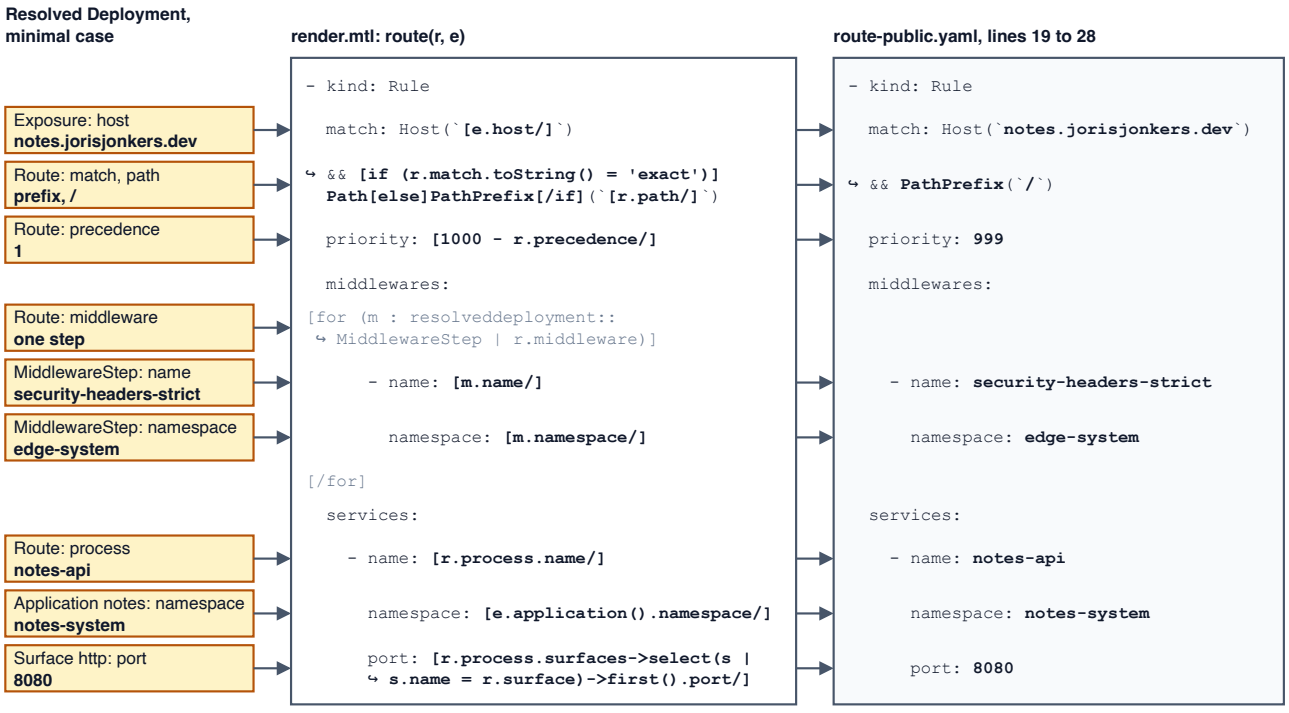


Figure 3: From Model to File

dashes. A Vault document is JSON with no header.

Entry point. The pipeline's entry point is `Pipeline.render` in the CLI bundle. It checks every input document. If validation rejects the set, it returns diagnostics and writes nothing; otherwise, it lowers and resolves each Project and calls `Rendering.render(deployments, root)` once. The pipeline has no command-line interface; it runs in the Maven build of `emf/`, where `RenderedTreeTest` compares each rendered file with the committed `rendered/` tree. The parity suite uses the same entry point exclusively through files (`emf/docs/adr/emf/0080-parity-crosses-the-cli-file-interface.md`). In Eclipse, `render.launch` (class `RenderModel`) and the hand-in's launch configurations render one model file through `Rendering.render`.

3.2 The Main Template and the Path Plan

The seven-line main template (Example 1) opens one file per Deliverable at the model's assigned path; it computes no paths.

```
[template public render(deployment : resolveddeployment::ResolvedDeployment)]
  [for (d : resolveddeployment::Deliverable | deployment.deliverables)]
    [file (d.path, overwrite, 'UTF-8')]
      [d.document()/]
    [/file]
  [/for]
[/template]
```

Example 1: The Main Template

The private template `document` chooses the format: JSON for the three Vault kinds and the catalog method's policy part, YAML for everything else. YAML generation calls `objects` on the Deliverable; Acceleo selects the overload for its runtime type, with one overload for each of the 38 YAML kinds.

Resolution defines the path plan; templates only follow it. Each Project has a namespace file and an index for `apps/<project>/`; each Application has one file per family, with no file for a family containing no objects. No index lists policy files or Vault documents, because the Vault policy Job reads them from its own ConfigMap (`spec/v1/30-deliverables.md`).

Several Resolved Deployments, one owner per file. A Project owns every file in its directory, so indexes are not shared and each lists only its own Deliverable's resources (Example 2). The composed minimal and data cases match the two committed trees file for file (Section 4).

```
[template private objects(d : resolveddeployment::IndexFile)]
  ---
  apiVersion: kustomize.config.k8s.io/v1beta1
  kind: Kustomization
  resources:
  [for (r : String | d.resources->sortedBy(r | r))]
      - [r/]
  [/for]
[/template]
```

Example 2: An Index Lists Its Own Directory

Sorting by value makes indexes independent of Deliverable order. The module previously collected indexes through `allInstances()`; this was removed with the estate pass (Section 2.2).

Two Deliverables assigned the same path would silently overwrite the earlier file; templates cannot detect this. The pipeline therefore checks the plan before rendering and the written files before moving them into place, and rejects any path, ServiceAccount or Vault name with two owners, reporting `E_PATH_COLLISION` (`spec/v1/30-deliverables.md`).

3.3 One Template per Deliverable Kind

The submitted module supports 41 Deliverable kinds in nine families (Table 2). Each YAML family uses an `objects` template overloaded on its receiver type. Acceleo selects the overload by runtime type rather than

string comparison: these are polymorphic templates in the course's sense. A kind with no overload causes evaluation and the run to fail. The module previously dispatched through 22 type tests with no `else`, so a kind without a branch would silently produce only a header line. Type dispatch closes this gap and is 50 lines shorter.

Family	Deliverable kinds	Output objects
Project files	`NamespaceFile`, `IndexFile`, `SourceFile`	Namespace; Kustomization; Flux `OCIRepository` and its Kustomization
Process workload	`WorkloadFile`, `SwitchoverFile`, `ServiceFile`, `BudgetFile`, `NodeAgentFile`, `PriorityClassFile`	Deployment (or declared controller); Flagger Canary and HorizontalPodAutoscaler; Service; PodDisruptionBudget; Node Agent DaemonSet; PriorityClass
Identity and access	`IdentityFile`, `RbacFile`, `NetworkPolicyFile`, `ClientFile`, `HumanGrantFile`	ServiceAccount; Role or ClusterRole and binding; NetworkPolicy; sign-in `Client`; bindings for human groups
Routing	`IngressRouteFile`, `AliasFile`, `SharedMiddlewareFile`, `StableAddressFile`, `AddressPoolFile`, `TlsStoreFile`	Traefik IngressRoute with rate-limit Middleware; `ExternalName` Service; shared Middlewares; Stable Address Service; MetalLB `IPAddressPool` and `L2Advertisement`; default `TLSStore`
Storage and configuration	`ClaimFile`, `LibraryFile`, `ConfigMapFile`, `BackupFile`	PersistentVolume and Claim; immutable ConfigMap per Asset; backup CronJob
Jobs	`MigrationFile`, `MoveFile`, `PolicyJobFile`, `CatalogJobFile`	Suspended Migration Jobs; Move Jobs with their Role; Vault policy and catalog Jobs by `part`
Observability	`MonitorFile`, `EventsFile`, `DashboardFile`, `GrafanaFile`, `AlertRulesFile`, `AlertingFile`, `FluxMonitorFile`, `K3sMonitorsFile`	ServiceMonitor or PodMonitor; Flux Provider and Alert; `GrafanaDashboard`; `Grafana`; `PrometheusRule`; `AlertmanagerConfig`; Flux's PodMonitor; k3s scrapes
Secrets	`SecretFile`, `ConnectionFile`	`VaultAuth`, `VaultStaticSecret`, `VaultDynamicSecret`; `VaultConnection`
Vault documents (JSON)	`VaultPolicyFile`, `VaultRoleFile`, `GeneratedKeysFile`	Policy, auth role and generated-key documents; no header

Table 2: The 41 Deliverable Kinds in Nine Families

`MoveFile` renders a Move handed to it, kept for the migration (Section 1.2). A Deliverable represents a file rather than an object. Policy Jobs and catalog methods are exceptions: the `part` attribute assigns each of their five objects to a separate file, so each is planned and pruned as a separate Deliverable.

Everything added since Task 2 followed the same sequence: a metamodel kind or feature, its derivation in resolution, a template, and a committed file for byte comparison. Features of existing kinds include Route annotations (facts about the Route's tier) and the fourth Canary webhook, `rollback`, through which the Release Gate decides whether to roll back a release it can no longer pass. Each new kind received one `objects` overload; the edge's shared Middlewares, for example, are named by Routes through `MiddlewareStep.name`.

Templates derive no values, but specify how the target format represents them. The module's comments cite the chapter defining each rule:

- **Update strategy.** Blue-green uses `RollingUpdate` with a surge of one and no unavailable pods, because Flagger starts the new version beside the old. Rolling uses a surge of zero and one unavailable pod, because a proxy using a host port cannot run two copies. Stop-start uses `Recreate`.
- **Disruption budget.** The selector is the resolution's `servingLabel` for a blue-green Process, naming the Deployment managed by Flagger (`serving`); otherwise, it selects the Process's own pods.
- **Route match.** `exact` is written as `Path`; every other match as `PathPrefix`.
- **Migration Jobs.** They are created suspended and named for their revision's tag. Only the Release Gate (the check that releases a migration) changes their suspension.

3.4 Shared Templates and the Whitespace Rules

A shared template has multiple callers, such as the template for a Process's metadata or label set. It is written once at the left margin; callers determine its placement. Three whitespace rules apply. MOFM2T §8.4 defines what templates write at the start and end of a line. Acceleo 4 adds two behaviours used by the module.

The indentation margin (Acceleo 4). Each block (the template, each `if`, each `for`) adds two indentation spaces that Acceleo omits from the output. A line inside n enclosing blocks therefore appears $2n$ spaces further left. A Deployment's `strategy` is indented six spaces in the module and two in the output. A negative fixture with no body margin must fail with an Acceleo parsing error (`emf/bundles/render/src/test/resources/unindented.mtl`).

The call column (Acceleo 4). A call repeats the text preceding it on the line before every output line, so a shared template need not know where it will appear. In Example 3, `metadata` calls `labels` at column two of its own output, and `labels` calls `labelSet` at column zero, so the five label lines appear at column four of the Deployment. The label set is written at the margin; its output indentation depends on the call column.

```
[template private labels(p : resolveddeployment::ResolvedProcess)]
  [p.name.labelSet(p.component, p.application().id)/]
[/template]

[template private labelSet(name : String, component : String, application : String)]
  app.kubernetes.io/name: [name/]
  app.kubernetes.io/instance: [name/]
  app.kubernetes.io/part-of: [application/]
  app.kubernetes.io/managed-by: deploy-kit
  app.kubernetes.io/component: [component/]
[/template]
```

Example 3: The Label Shared Templates

List items. A list item is a shared template that begins with its own `- `, and the caller does not write the dash: a dash before the call would be repeated before every line, making each line a separate item.

Whitespace is therefore a template concern (Section 5.3.2). In return, the layout is visible in the template and checked without an intervening formatter.

3.5 Writing YAML and JSON

Header. A YAML Deliverable begins with `# GENERATED. Never hand-edit.`, written once per file by `document`. A JSON document has no header, because a comment would make it invalid for the Vault job.

Quoting. A string is written as a plain scalar unless the `scalar` query matches an implicit-type pattern; matches are double-quoted. The patterns match the whole value and cover the empty string, tilde, null, the boolean words (`y`, `yes`, `on` and their opposites), integers in every YAML 1.1 base, floats and timestamps. YAML 1.1 rules are deliberate: Kubernetes tooling decodes through `sigs.k8s.io/yaml`, which follows YAML 1.1 implicit types, and the oracles are written under the same rules. A value such as `on` would reach the API server as a boolean if written as a plain scalar; unnecessary quoting costs only bytes. The string `50` is written `"50"`. Values a YAML reader would misread are also quoted: those opening on an indicator character, containing `: ` or `#`, ending in a colon, or opening or ending on a space. The edge's Content Security Policy, which ends in `https:`, is quoted in the committed tree.

The double-quoted form does not escape its content, so a quote or backslash in a value would break the line. The model's value rules are intended to exclude these characters.

Block scalars. An Asset is a file a Project ships as configuration. Its ConfigMap holds the text as a block scalar, and its name contains a digest of the content, so an edit produces a new immutable object (`docs/adr/model/0014-file-shaped-configuration-is-an-asset.md`). Two queries write it (Example 4).

```
[query private chomping(content : String) : String =
  if content.endsWith('\n\n') then '|+' else if content.endsWith('\n') then '|' else '|-'
    endif endif /]
```

```
[query private blockOf(content : String, indent : String) : String =
  (if content.endsWith('\n') then content.substring(1, content.size() - 1) else content
    endif)
    .replaceAll('(?m)^(?=.)', indent) /]
```

Example 4: The Block Scalar Queries

The indicator preserves trailing line breaks: one newline gives `|` (clip), two give `|+` (keep), none give `|-` (strip). The body drops one trailing newline, supplied by the block's own clipping, and indents every non-empty line by four spaces; blank lines stay blank.

Key order. Key order follows the template text. The API reads objects as maps, but the Acceleo 4 decision requires byte equality (decision record 0077 under `emf/docs/adr/`).

Vault documents as JSON. A policy document has one `path` member, with a `capabilities` list per path. A Kubernetes auth role has three lists containing one name each. The `json` templates pretty-print these documents with two spaces per level, using the margin rule, and escape list brackets with `['['/]`. The policy also appears as canonical one-line JSON in the Vault policy Job's ConfigMap, written during resolution and covered by the digest in the ConfigMap's name. Templates do not check that the two agree; parity comparison checks this only for committed cases. A generated-key document is the only JSON file built by concatenation: the `entry` query returns one key as an object, and `json` separates the entries with commas. The concatenation writes keys, paths and encodings without escaping, which is safe only while the model restricts them to characters that need none.

3.6 Queries: Formatting and Selection

The module contains 33 queries as submitted, counting each overload. Seven earlier queries were removed because they chose values, and the choices are now represented by model features: `allowsAt`, `policyPaths`, `catalogAllows`, `catalogPaths`, `middlewareName`, `rateLimitName` and `listed`. `serving` became `servingOf`. Table 3 lists the five choices behind them.

The remaining queries derive no model values. Formatting queries read one value and return a string or a number (`seconds`, `octal`, `fileOf`, `directoryOf`, `writableOf`, `scalar`, `chomping`, `blockOf`, `entry`). Selection queries determine which objects a template visits and their order (`application`, `routed`, `paths`, `hostnames`, `backups`, `backupRanges`, `files`, `destinationOf`, `versioned`, `versionOf`, `servingOf`, `operatedOf`, and `holdsEnv`, `mounts` and `probed` for a container's optional sections); the module's comments specify each order. The per-kind rules above (including `switchover` and `servingLabel`) specify how the target format represents model values without deriving them.

Each of the five choices produced a value beyond the reach of model-level tests. These values are now derived once during resolution and only written by templates. The trees remained identical byte for byte, except that a redirect step previously named as a security profile by `middlewareName` is now named for its Route, as in the oracles.

Choice	Formerly in the template	Now in the model
Default Vault capability	`allowsAt`: `read` for a path with no entry	`VaultPolicyFile.paths`
Catalog capabilities	`catalogAllows`: by the path's prefix	`ResolvedCatalogJob.policy`
Security profile name	`middlewareName`: by content policy	`MiddlewareStep.name`, `.namespace`
Rate limit name	`rateLimitName`: by the Route's position	`MiddlewareStep.name`, `.namespace`
Primary Deployment name	the `-primary` suffix, in two templates	`ResolvedProcess.serving`, `.servingLabel`

Table 3: The Decisions Moved from the Templates into the Resolution

The full list of changes is in Section A.4 and Section A.5.

3.7 The Java Class and Failure Handling

The Java class `Rendering` is the only Java in the step. It copies the Resolved Deployments into a fresh resource, loads the module from the class path, and runs the standalone generator in a `.partial` directory beside the

output root. If Acceleo reports a diagnostic other than OK, the class throws an `IllegalStateException` whose message begins `the rendering did not complete` and includes Acceleo's text. Files move under the root only after success, so a failed run leaves the root unchanged. A template that fails to parse or evaluate therefore cannot silently produce an incomplete tree, as it previously could.

3.8 Worked Example: One Route from Model to File

Figure 3 places the model values, the Route template and the committed YAML side by side. The minimal case has one Exposure serving `notes.jorisjonkers.dev`, with one Route matching the path `/` as a prefix and reaching the Process `notes-api` on its `http` Surface (`spec/v1/examples/minimal/notes.project.yml`). The host and the path are authored, and only `exact` would be written as `Path` instead of `PathPrefix`. Resolution assigns precedence 1; the template writes `priority: 999` by the arithmetic `1000 - r.precedence`, so that the smallest precedence has the highest priority. The middleware step carries its name and namespace, so the template needs no conditional choice. The module previously chose them by the step's kind, with a query for each name (Table 3). The port is looked up by the Surface's name among the Process's Surfaces, so the file states the port the Process declares.

4 Tests and Evaluation

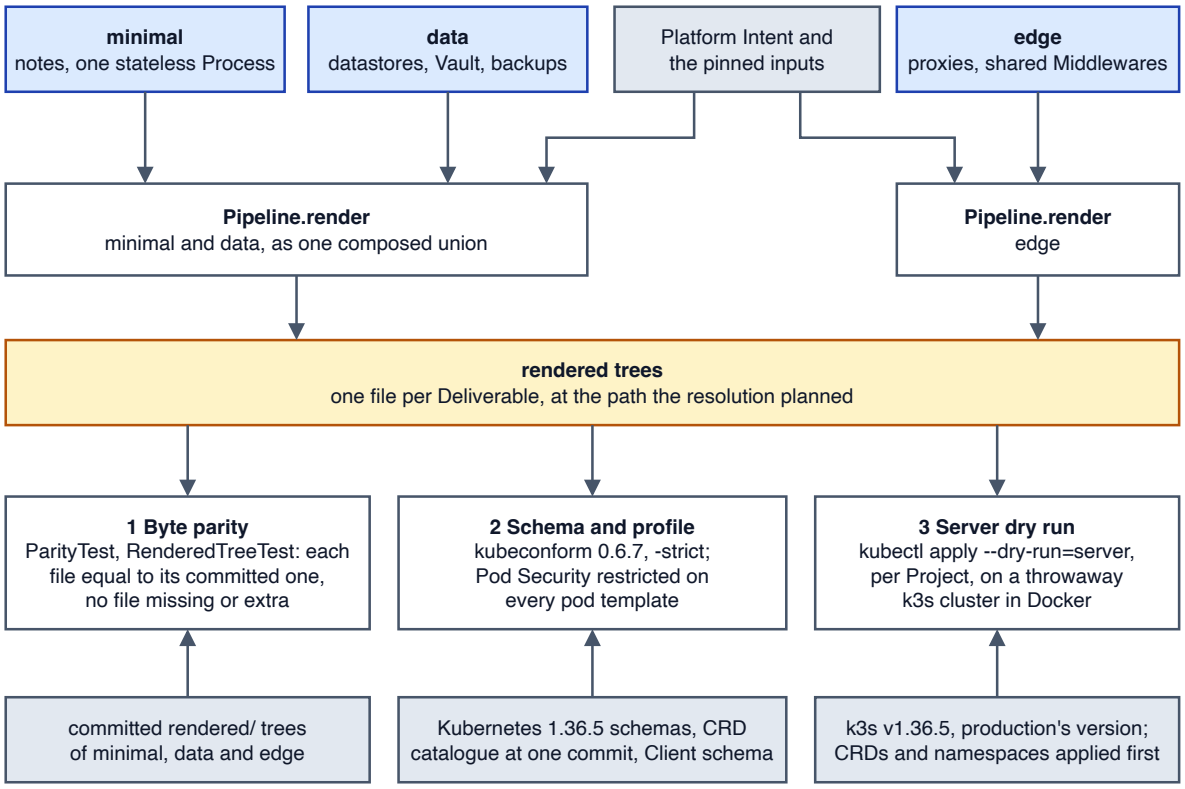


Figure 4: Checks from Generated Files to the Cluster

The evaluation asks whether each transformation writes the intended model or text and whether external consumers accept the result. Committed expected files answer the first question; schema validation, a security profile and an API server address the second. Figure 4 shows these checks along the path to the cluster.

4.1 Testing Strategy

Table 4 lists eleven checks at four levels. Each row names what the output is compared with and the defects it can expose. Passing one check does not rule out defects covered by another.

Check	Level	Compared with	Defects it can expose
Stage tests (`StageTest`)	1. Stage seam	Committed expected files of each stage, each the next stage's input	A wrong value in any stage's output, named by its path
Field coverage (`FieldCoverageTest`)	1. Stage seam	Every feature, literal and kind the metamodels declare	A feature no case exercises
Behaviour witnesses	1. Stage seam	85 model rows of the behaviour ledger	A model behaviour without a test
Seam tests (`RenderingTest`)	1. Template seam	Expected text written in the test	A wrong header, path or key order; a template that does not parse; files left by a failed run
Composed union parity (`ParityTest`)	2. CLI file interface	Committed trees of `minimal`, `data` and the platform's share	Any changed byte; a missing or extra file
Per-Project output (`RenderedTreeTest`)	2. CLI file interface	Committed trees of `minimal`, `data` and `delivery`	A file written into another Project's output
Determinism and rejection (`RenderedTreeTest`)	2. CLI file interface	A second run; a set the checks reject	Varying output; a partial tree on rejection
Oracle distance (`OracleDistanceTest`)	2. Pipeline	Every oracle under `spec/v1/examples/`	An oracle on the allowlist that no longer matches
Schema (`lint: manifests`, kubeconform)	3. External definition	Kubernetes 1.36.5 schemas, the datreeio CRD catalogue, the `Client` schema	Unknown or missing fields, wrong types
Pod Security (`scripts/lib/pod-security.ts`)	3. External definition	Pod Security `restricted` plus a read-only root filesystem	A pod template that drops a privilege control
Server dry-run (k3s cluster)	4. API server	Admission by a cluster using the production version	Objects accepted by the schema but rejected by the cluster

Table 4: The Eleven Checks, Their Levels, Oracles and Detectable Defects

4.1.1 Test Levels

Stage tests. Each transformation is unit-tested on its own: `StageTest` runs parse, lower, resolve and render in-process, each from committed input to committed expected files under `emf/bundles/cli/src/test/resources/stages/<stage>/<case>/`. Only `parse` reads authored input; every later stage reads the expected output of the stage before it, so one file is one stage's oracle and the next stage's input, and two stages cannot drift apart. A render case's expected files are copies of the committed oracles, held to them byte for byte. `FieldCoverageTest` holds each stage's cases to every feature, enumeration literal and template dispatch its metamodels declare; the gaps left are listed per stage in `gaps.txt`, a baseline that may only shrink. Each of 85 model behaviours of the behaviour ledger has a JUnit witness, and three more are listed as pending (`emf/docs/witnesses.md`). The build enforces 100% line and branch coverage (JaCoCo) and a 100% mutation score (PIT) per module and fails below either; the suite has 662 tests.

Seam and file interface. The seam tests in `emf/bundles/render` exercise `Rendering` without the pipeline. A template that does not parse must fail with Acceleo's diagnostic. A template that fails halfway (`halfway.mtl`) must leave the output root unchanged, as the sibling `.partial` directory guarantees. A deliberately unindented module (`unindented.mtl`) is the negative case for the indentation rule of Section 3. `ParityTest` in `emf/tests/parity` runs the whole pipeline through its command-line file interface (`emf/docs/adr/emf/0080-parity-crosses-the-cli-file-interface.md`) and checks that the composed union contains exactly the committed files, each identical byte for byte. `RenderedTreeTest` checks that each Project's output matches its own committed tree, that a second run writes the same bytes, and that a set containing an invalid document (`surface: grpc`, rejected with `E_UNKNOWN_SURFACE`) produces no files. `OracleDistanceTest` measures every oracle's distance from what the pipeline writes and holds 175 comparisons on an allowlist to an exact match.

External consumers. `scripts/lint-manifests.ts` runs kubeconform 0.6.7 with `-kubernetes-version 1.36.5` and `-strict` against the default schemas, the datreeio CRD catalogue and the repository's `Client` schema, and checks every pod template against Pod Security's `restricted` profile; both checks have negative fixtures. `handin/scripts/dry-run.sh`, run as `dry-run.sh output "" generated`, applies them to the generated tree and takes about ten seconds. It then starts a disposable k3s cluster at v1.36.5, the production version, installs pinned CRDs for every non-core kind, creates the namespaces the output names and applies each Project's output and the platform's share with `--dry-run=server`. An earlier attempt at 1.31 failed

because the `GrafanaDashboard` CRD requires a newer API server. Network policies are absent from indexes and are applied file by file.

4.1.2 Oracles and Their Independence

A committed file is an *oracle* in this report: the expected output against which a result is compared. The oracles under `spec/v1/examples/` are produced by the production implementation; the model-driven implementation is held to them, and its tests refuse to rewrite them. Appendix B discusses that implementation and the limits of this independence.

4.1.3 Comparison: Content First, Bytes Last

Every intermediate model and every output is structured data, so `Comparison` compares parsed content first and bytes last. JSON is read with Jackson, YAML with SnakeYAML's YAML 1.1 reader on both sides, as Kubernetes reads it, so a quoted `"on"` and a plain `on` still differ by type, and XMI is loaded into EMF and compared with its equality helper. Every difference is reported as a path into the document, with a list item named by its key: `.spec.template.spec.containers[name="vault"].command: missing (expected a list of 1)`. Only when the content agrees are the bytes compared, and a difference there is reported as "formatting only", naming the first line that differs. Byte identity remains the bar: a formatting-only difference still fails. A failure names the field at fault instead of a line diff over the whole file, which sped up debugging.

4.2 Example Models

The suite renders three examples: `minimal` (15 files, one Process with a blue-green `Canary`), `data` (76 files, five datastores and an asset-sync Process with Vault delivery) and `edge` (36 files, the two Traefik instances and a DNS controller). `minimal` and `data` render together, with the platform's four estate-scoped files, as a composed union of 95 files, checking that adding a second Project does not change the first Project's bytes; `edge` is compared separately.

4.3 Results

Table 5 reports each check for each case. The schema, profile and dry-run results were obtained on 2026-10-31 from the generated tree. Units differ by row: output counts refer to files; schema counts refer to resources (several may share a file, and `kustomization.yaml` files count); dry-run counts refer to objects; both include the platform's share. The profile check covers 19 pod templates; only parity reads the Vault JSON files.

	minimal	**data**	**union**	**edge**
Committed files (YAML, JSON)	15 (15, 0)	76 (64, 12)	95	36 (34, 2)
Written by the run	15	76	95	36
Byte parity	within union	within union	pass	pass
Per-Project output	pass	pass	–	–
Second run writes the same bytes	–	–	pass	–
Schema (kubeconform)	22/22 valid	95/95 valid	–	48/48 valid
Pod Security `restricted`	no finding	no finding	–	no finding
Server dry-run, indexed objects	15/15	72/72	–	35/35
Server dry-run, network policies	2/2	12/12	–	5/5

Table 5: Results per Check and Case

The API server issued two warnings for `edge`'s public Traefik `Deployment`: two entry points are each declared twice. This reflects a modelling choice rather than a defect.

4.4 Defects Found

Most rendering defects were found by running the output rather than by byte comparison. The trees changed alongside the implementations, so comparisons passed both before and after a fix. Five examples illustrate this:

- A pod template omitted `allowPrivilegeEscalation: false`: found by reading the profile; the Pod Security check now catches it.

- A Job's name did not reflect its pod template, so the cluster rejected it: found only on a cluster.
- Eligible nodes were derived but not rendered: found by reading the adapter; parity catches it now.
- A space in an authored path created a second Release Gate entry, the condition per Reconcile Unit that blocks a rollout: found by review; validation now rejects it.
- An optional edge between Projects made two Reconcile Units, the units Flux reconciles in order, wait for each other: found in use; no example declares such an edge.

4.5 Limitations

Template coverage. Comparisons provide no evidence about kinds absent from the compared trees; the render stage's baseline names each one. Acceleo template coverage is not measured: the gates do not reach templates, and no tool reports which branches the cases exercise. A one-character change to the Namespace template's `managed-by` label failed 14 of 25 stage cases and 5 of 9 rendered-tree tests, each naming the file; a change to a kind no compared tree holds would pass.

Version pins. Schema validation is pinned to Kubernetes 1.36.5 and one CRD catalogue commit, while production worker nodes run 1.35.2. A field ignored by a 1.35 kubelet would therefore pass validation.

Fresh-cluster dry-run. This cannot detect defects involving immutable fields; no controllers run and no network policies are enforced. The model-driven implementation does not apply objects, so these checks do not exercise Flux's ordering, pruning or release gating.

When the build runs. The `Model-driven build` job runs the Maven verify, tests and the coverage gate, on every commit that changes `emf/` or the workflow; other changes skip it. The mutation gate takes too long for every change and runs nightly instead. A Maven wrapper update once broke the Tycho build unnoticed while it ran only on request: an implementation stays level with shared oracles only while every change runs its build.

5 Reflection

The project ran from the Task 0 proposal in September to this submission on 30 October. Figure 5 places the three tasks, both presentations and the project's phases on one timeline, beside the commits per day.

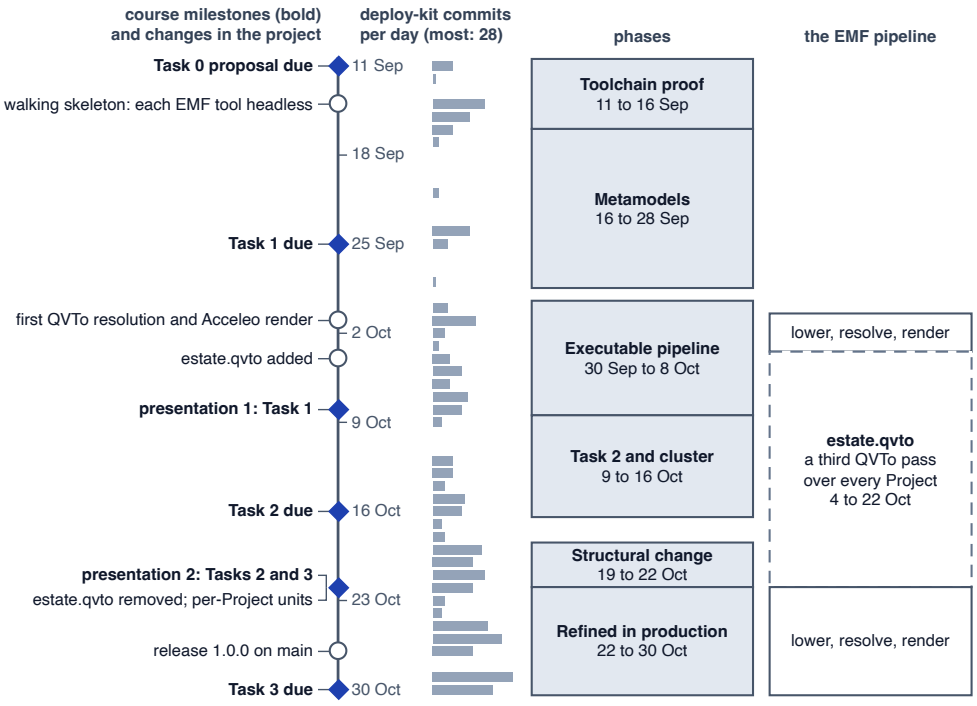


Figure 5: Project Timeline

5.1 How the Project Changed Over Time

The proposal fixed a closed set of inputs, an intermediate Resolved Deployment, a generated Deliverable Set and two implementations tested against the same examples, but left the build order open. Priorities shifted through six phases.

- 14 to 16 September: demonstrating that each tool could run, through the headless build, the oracle layout and the parity seam.
- 16 to 28 September: the metamodels, ending in the Task 1 submission on 25 September.
- 30 September to 8 October: the pipeline became executable, with the first byte-for-byte Acceleo rendering of `minimal` on 1 October.
- 9 to 16 October: the Task 2 report, written after the Task 1 presentation on 8 October.
- 19 to 22 October: the estate pass was removed (Section 2.2.3); the second presentation was on 22 October.
- 22 to 30 October: the model was refined in production and reached release 1.0.0; the model-driven implementation adopted each change. In October, the repository received 275 commits, 168 of which touched `spec/v1`.

Three factors drove these shifts: deadlines three weeks apart and then two, cases surfaced in production, and oracle changes the model-driven implementation had to follow.

5.2 What Went Well

5.2.1 Oracle Parity Across Two Implementations

Both implementations are held to the same committed oracles, and neither is generated from the other, so they converged on the same behaviour (Appendix B). Section 4.1.2 explains the limits of this evidence.

5.2.2 Specification and Decisions Before the Reports

The normative model and the decision records were written before the reports, so a report could cite the record justifying each decision. The rendered tree of `minimal` was committed on 8 September, three weeks before the first Acceleo template, so it specified the expected output independently of the templates.

5.2.3 A Headless EMF Toolchain

Maven with Tycho runs every EMF tool without an IDE, using one dated Eclipse release. A walking skeleton, metamodel, OCL file and template each ran through its own tool before substantive content was added. When the Acceleo module arrived on 1 October, subsequent failures therefore concerned content.

5.2.4 Output Tested Against a Live Cluster

Applying generated files to a real cluster exposed faults beyond parsers and unit tests: an immutable Deployment selector, Jobs requiring recreation, and an omitted Pod Security control. This motivated the server dry-run in Section 4.1.1.

5.3 What Could Have Been Better

5.3.1 Structural Changes After Task 2

The Task 2 report describes three transformations; the estate pass was removed four days after it was rewritten, so the report is a snapshot of a pipeline that no longer exists. The appendix records the difference, but the structure should have been fixed at the Task 2 deadline, with later structural decisions recorded as changes.

5.3.2 Acceleo's Whitespace Rules

Byte comparison directed much of the template work towards presentation. Acceleo 4 strips a margin from each block-body line; a call repeats the text preceding it on the line before every output line, so shared templates

are written without indentation (Section 3). The `GENERATED` header, key order and quoting rule were template decisions made to match the oracle. The effort concerned output appearance, not transformation semantics.

5.3.3 Scope

The scope was too large for one course. Between 14 September and 30 October, the repository received 341 commits across the specification, the production implementation and `emf/`, a second implementation of the whole compiler, and keeping the two level doubled every change. A smaller scope would have kept the parity, decision records and headless build, held the model at its Task 1 scope and documented later features as deferred.

5.3.4 Structural Comparison Came Late

For most of the project, output was compared only as bytes: a line diff across several hundred lines does not distinguish missing keys, extra keys and wrong values. The structured comparison of Section 4.1.3 names each difference's path; adopting it from the start would have saved much debugging.

5.4 What Was Learned

5.4.1 Decide in the Transformation, Write in the Template

Model-to-model transformations make decisions; templates write the results. When templates derive no values absent from the model, output errors can be traced either to the model or to target-syntax formatting. The model can then be tested independently of the template.

5.4.2 Metamodels as Contracts Between Stages

Typing each stage's model catches errors earlier and makes each stage replaceable: the estate pass was removed without changing the parser or the renderer. The cost is that each metamodel changes when its stage does.

5.4.3 The Cost of Running MDE Tools Without an IDE

Most of the effort in model-driven tooling went into running it rather than modelling. The module's POM changed in at least ten commits, and the indentation rule was established experimentally. Build configuration and documented tool behaviour therefore deserve at least as much attention as metamodels.

5.4.4 Compare Parsed Documents Before Bytes

Oracles should be compared as parsed documents first, then as bytes, and each stage tested on its own: a failure then names the stage and the field at fault (Section 4.1.3).

6 Outlook

The project continues in a new repository with one Rust codebase for the compiler and in-cluster service. The web interface remains in Vue and uses the Rust API. The oracle files and parity method carry over: the Rust compiler must match the committed files under `spec/v1/examples/`, and a difference between implementations remains a defect in one of them. The model-driven implementation (`emf/`) will be retired after the grade is recorded; it served as the model-driven reference against which the oracles were first checked.

References

[1] Object Management Group. *MOF Model to Text Transformation Language (MOFM2T)*, version 1.0, 2008. https://www.omg.org/spec/MOFM2T/1.0/.

[2] L. M. Rose, R. F. Paige, D. S. Kolovos and F. A. C. Polack. The Epsilon Generation Language. In *Model Driven Architecture – Foundations and Applications (ECMDA-FA 2008)*, LNCS 5095, Springer, 2008.

A Changes to the Task 1 Metamodels and Task 2 Transformations

This appendix lists changes to the Task 1 metamodels and Task 2 transformations in the Task 3 pipeline, organised by artefact. The baseline is the Task 2 submission's metamodel appendix and design section. These describe three QVT-Operational transformations (lowering, resolution and the estate pass over the Resolved Deployments) and the Task 1-to-Task 2 metamodel changes. Each subsection contains tables with one row per change, identifying the affected classes and features and briefly explaining the reason.

The model was refined as it was applied to production: deploying the real estate surfaced cases the model had to express, and the changes below came from those cases. The tables group them by theme; changes that concern only the model-driven implementation, not the model, form a group of their own. The change set was derived from the repository's git history rather than a comparison of complete models, so each row summarises a commit's effect.

File sizes indicate the extent of the changes. As submitted, the Project Intent metamodel, which also holds the Platform Intent classes, has 137 classes and 441 features, the Resolved Deployment metamodel 133 classes and 619 features, and the Pinned Inputs metamodel 15 classes and 67 features. The resolution transformation has 4276 lines, 10 mappings and 305 queries, and the lowering has 473 lines and 54 mappings, compared with about 1700 and 340 lines in the Task 2 design section. The 134-line estate pass was removed. The Acceleo module has 3772 lines.

A.1 Source Metamodel (Project Intent)

The source metamodel defines what an author writes in a Project's files. Figure 6 shows its Project Intent part. Changes fall into five themes: how a `Process` reaches platform services, how it receives secrets, the identity and placement constraints checked by the model, further forms of `Process`, and the exposure, sign-in and observation of an `Application`. The Platform Intent classes live in the same `project-intent.ecore` file, and Section A.2 covers them separately.

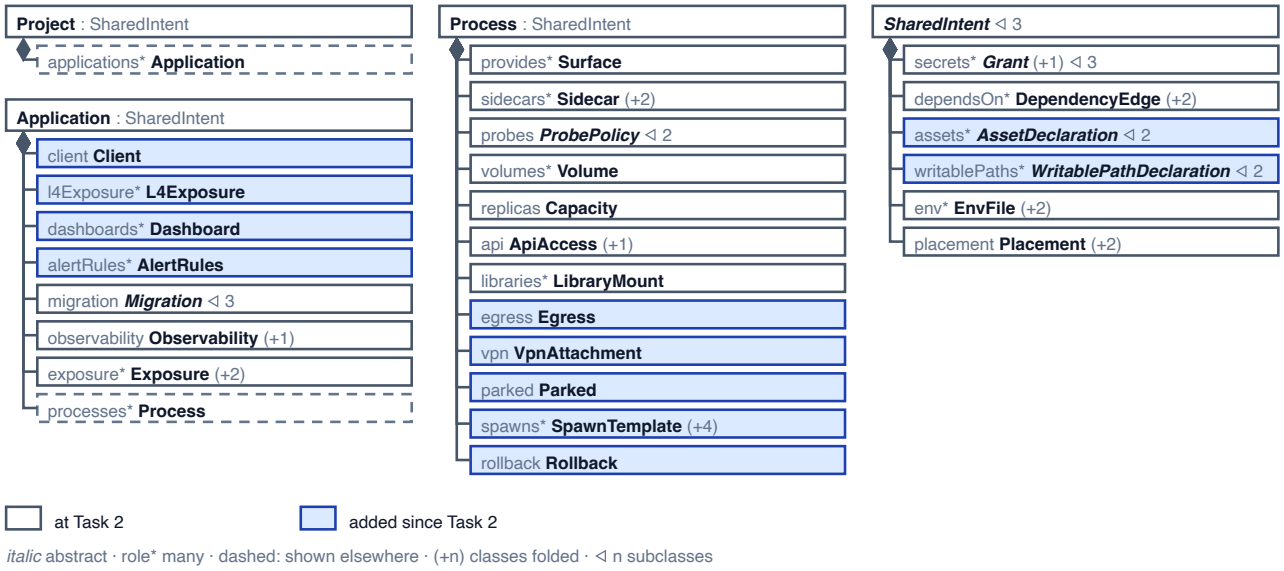


Figure 6: The Source Metamodel as Submitted

These changes keep authored documents small. Each new field records a choice the author must make or a reason the model could not infer. No field is added where a platform fact already determines its value.

The OCL constraints also changed substantially. The constraints above are among them, and each new form of `Process` brought its own constraints into `project-intent.ocl`.

A.2 Platform Intent

The platform team writes one Platform Intent for the estate. It defines choices unavailable to Projects: privileged roles, engines and their catalog methods, images for platform components, resources available to Processes and the policies limiting their use. Figure 7 shows its classes. Its metamodel is the Platform part of `project-intent.`

Change	What changed	Why
Reaching the platform's services		
Process VPN attachment	`Process` gains `vpn`, a `VpnAttachment` holding one `forwardedPort`. Its image, credential and resource quantities come from Platform Intent.	The author specifies only the forwarded port, without tunnel details.
Internet egress	`Process` gains `egress`, containing `to` entries of `EgressTarget` and a required `reason`.	Default deny remains the rule; each exception has a declared reason for review.
Library mount	`Process` gains `libraries`: `LibraryMount` (`library`, `mountAt`, `subPath`) with a `LibraryAccess`.	A library is a directory on one node. The mount names it and requests access, which is checked against the grant.
GPU placement	Placement gains a structured `gpu` with a vendor and model.	One declaration selects and requests a card, so the request is derived without repetition.
Secrets and keys		
Declared secret keys	The flat key list becomes `KvKey`, a `name` with either `PlainKey` or `GeneratedKey`. `GeneratedKey` holds `generate`: `bytes` and an `encoding` of `KeyEncoding`.	A platform-generated key is declared by its shape; its value never enters the model.
Credential delivery	A grant's delivery target gains `mountAt` and `fileMode`, allowing an edge's credential to be delivered as a file with its own mode, alongside environment delivery.	A Process may require file delivery; the file's location and mode form part of the contract.
Transit read	A transit grant may declare `read`, covering the key's public part.	A JWKS endpoint needs only the public part.
Identity and constraints		
Consumer addresses by Application Id	An edge names its provider by Application Id, checked by the `DependencyEdges` descriptor and an OCL constraint.	Consumer configuration depends only on the Id, never on a namespace or Project.
Unique claim reservation	An OCL constraint rejects a `claim` reserved by two volumes in one Project.	A Project's Processes share a namespace, so two volumes using one claim would silently share data.

Table 6: Changes to the Project Intent Metamodel Since Task 2: Reaching the Platform's Services; Secrets and Keys; Identity and Constraints

Change	What changed	Why
Command and memory-backed paths	A Process and a sidecar may declare `command` and `args`, lists of literal words replacing the image's entrypoint and arguments; a writable path may be `{path, memory}`.	Image startup concerns only its Process, so words cannot contain placeholders; memory-backed paths count towards that Process's memory.
API access scope	`ApiRule` is scoped to the Process's namespace or the cluster. A Process can access the Kubernetes API without a rule when its declaration suffices.	Access is declared at the required scope and no wider.
Spawn templates	A Process may create pods only from a declared spawn template.	Dynamic workloads are restricted to declared shapes.
Sidecar volumes and init	A `Sidecar` mounts the named Process volumes and may run before the main container as an init step.	A sidecar needs access to its data, with execution order declared.
Lifecycle: scheduled and parked	`lifecycle` may be `scheduled` with its own cron schedule; a Process may be `parked` with a `reason`.	A scheduled Process renders a CronJob; a parked Process retains its claims and grants without running pods.
Capacity across nodes	A `Capacity` exception may specify `across: node`.	Replicas spread across nodes without a second rule.
Low port	A Process may listen below 1024.	The substrate binds any port without a capability, preserving the restricted posture.

Table 7: Changes to the Project Intent Metamodel Since Task 2: Process Forms and Their Constraints

Change	What changed	Why
Raw Surfaces	An `Application` gains an `l4Exposure` (port, protocol, address, Process, Surface); a Surface may use a node's own address.	Raw TCP and UDP services receive a declared address.
Route rate limit	A Route gains `RateLimit` with `requests` and `per`.	A Route may limit requests per client; the proxy enforces this limit.
Route redirect	A Route gains `redirectTo`, rendered as a separate Middleware.	A redirect belongs to its Route.
Client block	An `Application` gains a `client` block with the sign-in fields required by its mode.	Sign-in is declared by the Application using it.
Dashboards and alert rules	The `Application` declares its Grafana dashboards and alert rules.	Observation is declared alongside the observed Application.

Table 8: Changes to the Project Intent Metamodel Since Task 2: Exposure, Sign-in and Observability

`ecore`; its grammar is `PlatformIntent.xtext`.

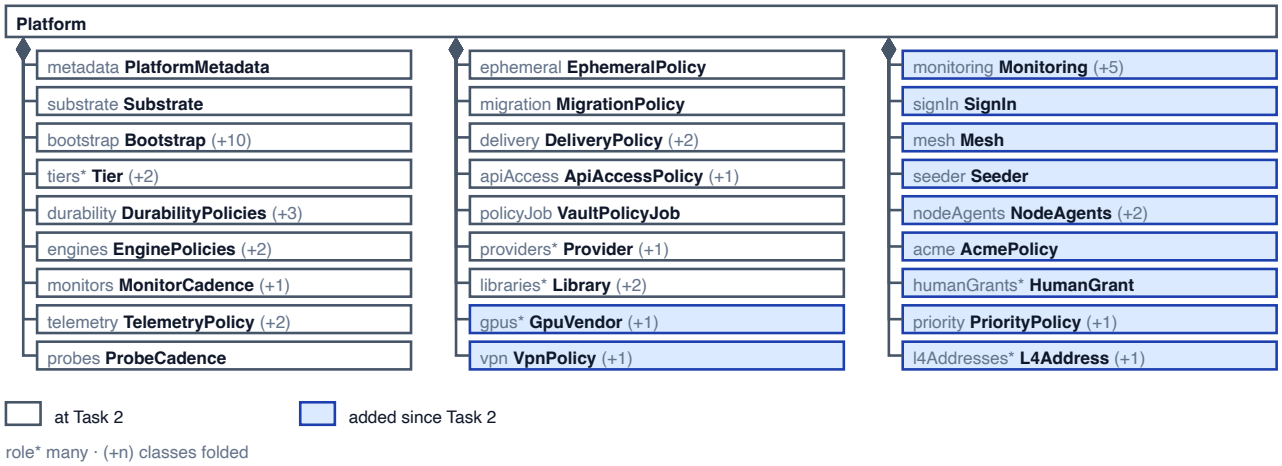


Figure 7: The Platform Intent Metamodel as Submitted

Platform Intent changed in two ways. First, the eight privileged-role fields now name both a Project and an Application, breaking compatibility with every document that sets them. Second, it gained policies previously held in transformations or Project files: the VPN image, GPU resource names, library grants and each engine's catalog method.

A.3 Pinned Inputs

Pinned inputs are documents read during resolution but not written by an author: the images lock, node contract and ClusterState snapshot. Their classes are defined in `pinned-inputs.ecore` and shown in Figure 8. The changes concern the images lock's versions and checks, and the snapshot's backup list.

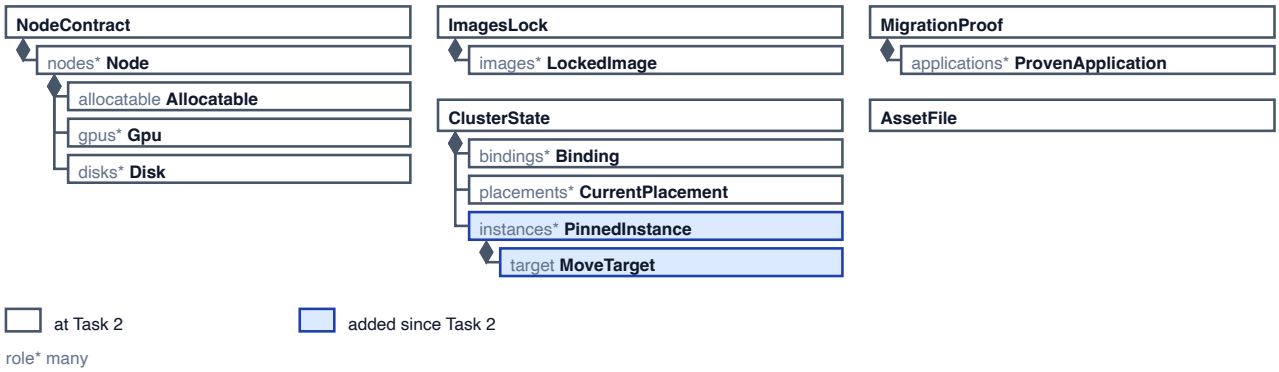


Figure 8: The Pinned Inputs Metamodel as Submitted

Change	What changed	Why
Privileged roles and human access		
Roles name their Project	The eight fields giving an `Application` a privileged role take `<project>/<application>`, resolved in `ProjectIntentScopes.java`. This breaks compatibility with every Platform Intent.	An Application Id can be declared in any Project's file; a bare Id cannot identify its Project.
Human grants	`humanGrants`: each entry contains a `group` (as seen by the API server), one of four built-in cluster roles, and a `reason`.	The platform decides who administers the cluster, so grants belong to no Application.
Engines		
Catalog method	An engine entry names a `catalog` method: an image that applies a catalog through the engine's Surface, with its user and sign-in.	Applying a database catalog writes to a live datastore using a privileged identity. The platform specifies each engine's method; the transformation only plans the Job.
New engines	`mongo` joins the engine vocabulary, and `s3` gains a bucket catalog method.	No catalog is derived for a store without databases; object-store keys and buckets are derived from edges.
Resources available to Processes		
VPN policy	`VpnPolicy`: an image, a credential, `memory` and `cpu`, and `VpnSetting` key-value entries.	The tunnel's image and settings are platform facts that no Project should set.
GPU vendors	`GpuVendor` with `GpuVendorPolicy`: a `resource` name and a `runtimeClass`.	The resource name and runtime class belong to the driver the platform runs.
Libraries	`LibraryDeclaration` with `Library` entries: a directory on one node and the Projects granted access.	The platform names directories and grants access; Projects mount only those granted to them.
Substrate pulls	The substrate's image pull is allowed within every derived deadline.	A deadline derived for another purpose must not cause an image pull to fail.

Table 9: Changes to the Platform Intent Since Task 2: Privileged Roles and Human Access; Engines; Resources Available to Processes

Change	What changed	Why
Tier forward auth	A tier's `forwardAuth` names a provider Application, its responding Surface and the path.	The endpoint follows the provider's Process, so moving the provider updates the edge's target.
Tier certificates and addresses	A tier gains an L4 Address and its own certificate; a Route takes its TLS from its tier.	Each tier is a proxy with its own entry point and certificate.
Tier DNS and ingress class	A tier declares `dns` (`target`, `proxied`) and an ingress class.	Routes carry the annotations required by public records and the selected proxy.
Default certificate	A tier that issues no certificate names a Secret Store path in `certificateFrom`.	Every proxy serves that certificate to a Route naming none of its own.
L4 Addresses	Named L4 Addresses, each an IPv4 pool announced at one site.	A raw Surface needs a platform-owned address.
Mesh enrolment	`mesh.enrolled` names the Projects in the ambient mesh.	Enrolment is the platform's decision, and a name no Project declares is rejected.

Table 10: Changes to the Platform Intent Since Task 2: Tiers, the Edge and the Mesh

Change	What changed	Why
Flux metrics	The bootstrap set's Flux entry records the port and path where its controllers serve metrics.	Flux serves all Projects and belongs to none, so the platform records how to reach it.
k3s metrics and monitoring stores	The bootstrap set's `k3s` entry (`K3sInstall`, `K3sMetrics`) records where the kubelet, the API server and the cluster DNS serve metrics; `Monitoring` gains `stores`, the Loki, Tempo and Pyroscope Applications Grafana reads.	The cluster's own components and Grafana's data sources belong to no Project, so the platform records them.
Priority classes	Two classes, `data` and `critical`, and a list of critical Applications.	Priority is derived from the Process rather than authored by it.
Node agents	Node agents are derived from node facts and pools.	Agents follow existing nodes without a duplicated list.
Foundation and bootstrap set	The remaining foundation and its bootstrap set are declared.	The platform specifies what a cluster needs before any Project runs.
Off-cluster recipient	Each off-cluster copy is encrypted to a platform-named recipient.	Only the holder of the recipient's key may read the copy.
DNS resolution	A DNS-serving Process resolves through the platform's upstream nameservers.	The resolver's upstream is a platform fact.

Table 11: Changes to the Platform Intent Since Task 2: Scheduling, Priority and the Foundation

Change	What changed	Why
Images lock		
Version on every entry	Each images-lock entry gains `version`, a `String` in `pinned-inputs.ecore` and `ImagesLock.xtext`. Rendering writes it on every pod running the image.	A pod must identify its software release.
Rejection of missing versions	The grammar accepts an entry without `version`; resolution rejects it as `E_IMAGE_VERSION_MISSING`.	This identifies the model entry, whereas a grammar error would identify only the line.
Root rejected	An image locked with uid 0 is rejected as `E_IMAGE_RUNS_AS_ROOT`.	The hardening posture requires a non-root uid, which a root image cannot supply.
ClusterState snapshot		
Backups removed	The snapshot's rollback-backup list and annotation are removed.	Rollback uses scheduled backups, so the snapshot no longer records an unused backup.

Table 12: Changes to the Pinned Inputs Since Task 2: Images Lock; ClusterState Snapshot

Output remains a function of the pinned inputs. A fact about the running cluster therefore enters as a new pinned value, rather than a live-cluster read. Section 3 covers the snapshot rules and the reason the node contract records capacity; they are not repeated here.

A.4 Target Metamodel (Resolved Deployment)

Resolution produces the Resolved Deployment; rendering reads it. It grew in two areas: values derived per Project and features required by workloads. Figure 9 shows the metamodel. As in Task 2, there is one root instance per Project, but its values now belong to that Project. In Task 2, every Project recorded the same Vault policy Job and rollout events. Each now has its own values, delivered by its own Reconcile Unit.

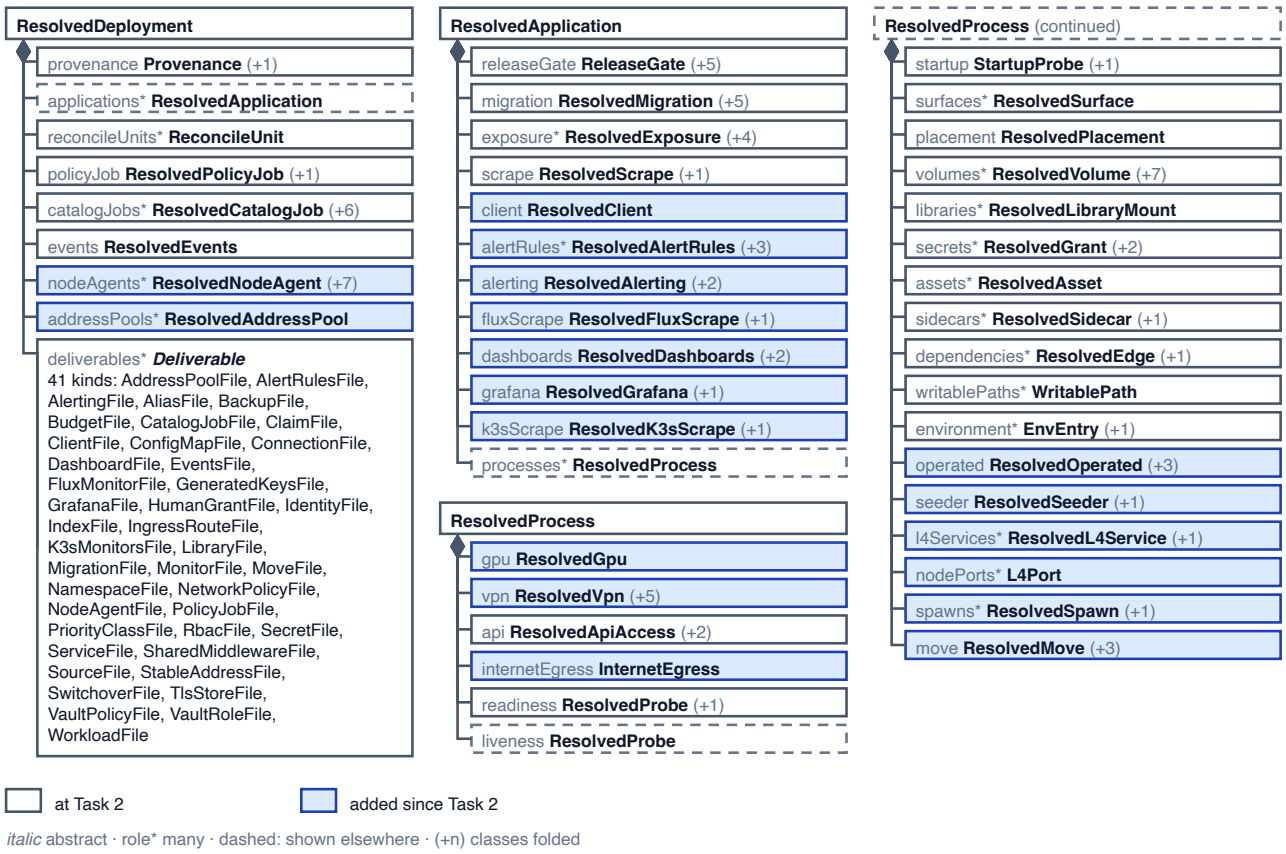


Figure 9: The Target Metamodel as Submitted

Every value needed for rendering is recorded in the Resolved Deployment, which is the only rendering input. Acceleo templates therefore read the consumer label, version and catalog identity from the model without deriving them.

A.5 The Transformations

The pipeline changed from Task 2's lowering, resolution and estate pass over several Resolved Deployments, followed by rendering, to lower, resolve and render. The estate pass was removed; resolution now plans its values per Project. Its ADR, `emf/docs/adr/emf/0076-qvto-derives-the-resolved-deployment.md`, was rewritten in place rather than amended, recording the current decision. It rests on the rule that a Project owns every object it delivers (`docs/adr/model/0116-a-project-owns-every-object-it-delivers.md`). The record accepts the cost: values needed by several Projects are planned separately by each.

Each change was made in the specification and in both implementations, without generating either implementation from the other. Committed oracle files hold both to the same result (Section 4); this appendix only records the changes.

Change	What changed	Why
Values planned in the Project's own Reconcile Unit		
Policy Job and rollout events	`ResolvedDeployment` retains `policyJob` and rollout events, now containing the Project's own Job, Flux provider and alert.	Each Project's Reconcile Unit writes its policies and reports its rollouts without another Project's values.
Catalog Job	`ResolvedCatalogJob` (`identity`, `namespace`, `image`, `imageVersion`, `uid`, `gid`, `role`, `address`, `memory`, `cpu`, `deadline`, `eligibleNodes`) and `ResolvedCatalog` (`host`, `port`, `connection`, and `CatalogDatabase` entries of `name`, `owner` and `data`).	A datastore's catalog Job is planned in the Project running the datastore. Its identity fields mirror the policy Job's.
Generated keys	`GeneratedKeysFile` is rendered as JSON by the policy Job.	The policy Job reads generated values from a file written during rendering.
Consumer admission and Stable Addresses		
Consumer label	A consuming Process has `consumes`, the label of each Surface it reaches; the provider's policy admits that label.	Only the new consumer is re-rendered, since the provider's policy already admits the label.
Stable Address	A resolved Stable Address holds its `name`, namespace, `target` and `port`. Each alias points to it through an `externalName`.	Consumers do not know where providers run, so relocating a provider changes one object.

Table 13: Changes to the Resolved Deployment Metamodel Since Task 2: Values Planned in the Project's Own Reconcile Unit; Consumer Admission and Stable Addresses

Change	What changed	Why
Workload features		
Image version on pods	Processes, sidecars and Jobs have `imageVersion`.	Rendering writes the version on every pod.
VPN	`ResolvedVpn` with `image`, `imageVersion`, `memory`, `cpu`, `forwardedPort`, `environment` and the `credential`.	The attached Process's tunnel container needs its own image and credential.
GPU and libraries	`ResolvedGpu` (`resource`, `runtimeClass`) and `ResolvedLibraryMount` (`library`, `claim`, `volume`, `node`, `path`, `gid`, `mountAt`, `subPath`, `access`).	The scheduling node and the mounted directory are facts of the pod, not of the Project.
Internet egress	`InternetEgress` records the `reason` for the exception.	The network policy places each reason beside its rule.
Release Gate provider	`GateProvider` records the Process a gate member waits for and the runner's version.	A gate member starts only after its required Processes in the same Project.
Process forms and tiers		
Command, memory-backed paths and Flux metrics	`ResolvedProcess` and `ResolvedSidecar` gain `command` and `arguments`; a writable path gains `memory`; `ResolvedApplication` gains `fluxScrape`.	Each resolves a declaration added to the source.
Process forms	`scheduled` (a CronJob), `parked`, `across` anti-affinity, `spawn` templates and the sidecar's `init` step.	Each form renders a different workload, and the target records which one it is.
Tier entries	The forward-auth endpoint, the Route's redirect Middleware, ingress class and DNS annotations, L4 Service, Client object and human-grant ClusterRoleBinding.	Each entry represents a tier or grant fact in Platform Intent.
Priority and node agents	A Process's priority class and the node agents' pools.	Priority is derived; agents are derived from node facts.
Monitoring stack and default certificate	`ResolvedOperated` (with `OperatedAlertmanager` and `OperatedFlag`) records a Prometheus or Alertmanager the Prometheus operator runs; `GrafanaFile` with `ResolvedGrafana`, `K3sMonitorsFile` with `ResolvedK3sScrape`, and `TlsStoreFile` carry the Grafana instance, the scrapes of k3s and a tier's default certificate; `ResolvedMemberlist` the node speakers' shared key.	Each object is derived from a platform fact, so the templates only write it.

Table 14: Changes to the Resolved Deployment Metamodel Since Task 2: Workload Features; Process Forms and Tiers

Change	What changed	Why
Capabilities, names and suffix	`VaultPolicyFile.paths` and `ResolvedCatalogJob.policy` record each path and its capabilities; `MiddlewareStep` gains `name` and `namespace`; `ResolvedProcess` gains `serving` and `servingLabel` (Table 3). The rendered trees keep their bytes, save one redirect step's name (Section 3.6).	A value chosen in a template lies beyond model-level tests.

Table 15: Changes to the Resolved Deployment Metamodel Specific to the Model-Driven Implementation

Change	What changed	Why
Pipeline		
estate pass removed	`estate.qvto`, its runner and tests are removed; the pipeline runs lower, resolve and render. Each Project's resolution plans the values previously planned by the pass.	With each Project owning its objects, no transformation needs to read several Resolved Deployments together.
Values planned and applied in the Project's own Reconcile Unit		
Vault policy Job	Resolution plans a Job per Project, in its namespace, named by the digest of its own documents.	A Project's Reconcile Unit becomes ready without waiting for a shared Job.
Rollout events	Resolution plans a Flux provider and alert per Project, in Flux's namespace, applied by that Project's Reconcile Unit.	Each Project reports its own rollouts.
Catalog Job	A datastore's catalog Job, ConfigMap, CronJob and policy are planned into its Project's output, under the Application running it.	The Reconcile Unit that starts the datastore applies its catalog without waiting for another Reconcile Unit.
Routes	Each Exposure's Route is written into its Application's directory and applied by its Project's Reconcile Unit, after the Reconcile Unit of the Project declaring the tier's proxy.	A Route depends on its named proxy; the Reconcile Unit graph specifies the order.
Job names and recreation	A catalog or policy Job is named by the digest of its inputs and pod template. If only rendering changed the template, the Job is marked for recreation.	Job templates are immutable, so a rendering change retaining the name would stall the Reconcile Unit.
Retries and placement	Policy and catalog Jobs retry until an attempt succeeds, with a deadline per attempt, and avoid nodes with SD cards.	A slow disk must not make a slow start permanently fail the Job.
Consumer admission by label		
Consumer label	Resolution gives each consuming Process the label of each Surface it reaches. The provider's policy admits the label from any namespace; the catalog Job holds its datastore's Surface label.	Adding a consumer re-renders only that consumer, leaving provider policies unchanged.

Table 16: Changes to the Transformations Since Task 2: Pipeline; Values Planned and Applied in the Project's Own Reconcile Unit; Consumer Admission by Label

Change	What changed	Why
Generated keys	The Vault policy Job generates each declared key and rejects generated paths escaping its Project's prefix.	The Job owning the path writes the generated value.
VPN and library	Each attached Process receives its own VPN credential; library mounts are resolved against the library's node.	Credentials are not shared between Processes, and mounts need their node's path.
File backups	A file backup runs as the Process owning the volume.	The backup reads the volume using its owner's identity.
Edges and ordering	A `required: false` edge imposes no Reconcile Unit ordering. A gated member starts only after its required stop-start Processes in the same Project.	Optional edges are not dependencies; a gate must not start before what it gates.
Secret Store connection	A datastore's Secret Store connection is named after its Process, not its engine.	Two datastores of one engine must not share a connection.
Images	Every locked image's version is read and written on its pods. Every derived deadline allows time for a pull.	Version is a pod fact; no deadline may prevent a pull scheduled by rendering.

Table 17: Changes to the Transformations Since Task 2: Resolution Rules

Change	What changed	Why
Rendering		
Restricted posture	The restricted posture sets `allowPrivilegeEscalation` to false on every container.	Pod Security's restricted profile requires this; otherwise, a setuid binary could escalate privileges.
Grant sync objects	A removed grant's sync objects outlive the release removing it.	A running Process may still read the synced secret during rollout, so these objects must outlive the release.
Writable paths	Every sidecar mounts the Process's writable paths.	A sidecar that reads the Process's writes must see them.
Rate limit and VPN templates	A Route's rate limit renders as a separate Middleware; a dedicated template renders a Process's VPN container.	Each is a separate object required by the proxy or pod.
Specific to the model-driven implementation		
Choices derived outside templates	`middlewareOf` names each step and sets its namespace; `servesFromPrimary` sets the serving Deployment and its label; `allowedPaths` and `ResolvedCatalog::policy` build each policy's paths, also used to write the policy Job's documents. Seven queries leave `render.mtl`: `allowsAt`, `policyPaths`, `catalogAllows`, `catalogPaths`, `middlewareName`, `rateLimitName` and `listed`.	Template choices were tested only through bytes. Derived values are model features covered by resolution tests; the policy Job and template read one list rather than deriving it separately.
Kind dispatch and failure	YAML kinds dispatch by type rather than a chain of type tests. Rendering writes files to a separate directory and moves them under the root only after all files are written.	A kind without a template fails the run; failure leaves the root unchanged.

Table 18: Changes to the Transformations Since Task 2: Rendering; Changes Specific to the Model-Driven Implementation

Change	What changed	Why
Stable Addresses		
Stable Address	Each Surface named by an edge or a tier's forward auth receives a derived Stable Address, rendered as one `ExternalName` Service. Aliases point to it.	Relocating a provider changes one object, without changing consumer configuration.
Process forms and the Release Gate		
Process forms	Scheduled Processes render a CronJob forbidding overlap; parked Processes retain claims; replicas spread across nodes; spawn templates bound the pods a Process creates.	The Process declaration fixes each form's behaviour, rather than a rendering flag.
Release Gate decisions	The Release Gate decides whether to roll back a release it can no longer pass.	The observing gate decides whether a release can still pass.

Table 19: Changes to the Transformations Since Task 2: Stable Addresses; Process Forms and the Release Gate

Change	What changed	Why
Flux metrics and metrics egress	The metrics stack's Project renders one PodMonitor for Flux in its own namespace; egress is derived to every declared scrape port and Flux's port. A memory-backed path adds its size to the Process's request, limit and eligibility.	The monitor and egress follow declared facts without manual specification.
Human grants	Each grant renders one ClusterRoleBinding from its role to its group, named for both.	Grants belong to no Project, so rendering or removing a Project cannot remove them.
Tiers	The tier entry determines its proxy configuration, forward-auth Middleware, shared Middlewares, Route TLS and DNS annotations.	A Route reaches exactly one proxy, selected by its audience.
Monitoring stack	Resolution derives the operator's objects for Prometheus and Alertmanager, the Grafana instance, the scrapes of k3s with their token and read access, and each tier's default TLS store.	Each follows a Platform Intent fact rather than a Project's declaration.
Encryption and resolver	Off-cluster copies are encrypted to the platform's recipient; DNS-serving Processes use the upstream nameservers.	Rendering uses both platform facts as given.

Table 20: Changes to the Transformations Since Task 2: Platform Policies and Tiers

B The Production Implementation

The production implementation, in TypeScript under `src/`, compiles the same model for production use. It renders the same cases, and `test/model/render.test.ts` compares its output with the same committed files. Its serialiser writes YAML with `version: "1.1"` (`src/infrastructure/serializer.ts`), the rule the templates' quoting follows (Section 3). The repository forbids generating either implementation from the other, and regenerating an oracle in CI. Where the two disagreed, the difference was a defect in one of them: each time the production implementation gained a rule, the model-driven implementation adopted it until both matched the oracles.

This independence is partial: one author wrote both implementations, and the committed trees changed in the same commits as an implementation. A self-generated snapshot accepts defects present when it was taken; a shared expected tree exposes disagreements between implementations. Agreement rules out errors made by only one implementation, but cannot rule out a shared misreading of the specification. External checks address part of this remaining risk: schemas, profiles and clusters check Kubernetes requirements rather than the project's specification. Each check has limits: byte equality establishes expected text rather than correctness; schemas cannot establish that a `NetworkPolicy` permits the intended traffic; dry-runs reveal nothing about controller behaviour. The Pod Security check permits one named exception: the VPN sidecar.